

The ARM Cortex-M4 Programmer's Model

Introduction to ARM Cortex-M Microprocessors – Chapter 2

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

- 1** Why This Matters
- 2** Architecture and Organization
- 3** ARM and the Cortex Profiles
- 4** STM32: The Chip and the Boards
- 5** Cortex-M4 Programmer's Model
- 6** Design Challenge

- 1** Why This Matters
- 2 Architecture and Organization
- 3 ARM and the Cortex Profiles
- 4 STM32: The Chip and the Boards
- 5 Cortex-M4 Programmer's Model
- 6 Design Challenge

The previous chapter introduced a generic processor. To write real programs, we need to know one specific processor in detail: how many registers it has, what they are called, which ones have a special meaning, and how it stores the result of a comparison.

This detailed view is called the *programmer's model*. It is the part of the hardware that software can actually see. Everything you write, in assembly or in C, works by using the small set of registers this chapter describes.

After completing this chapter, you should be able to:

1. Distinguish instruction set architecture, microarchitecture, and platform architecture, and give one meaning of “32-bit” under each layer.
2. State the defining characteristics of a RISC load-store architecture and contrast them with CISC.
3. Explain how a pipeline raises throughput, and why a short pipeline suits an embedded processor.
4. Select the appropriate ARM Cortex profile (A, R, or M) for a given application.
5. Name the Cortex-M4 core registers R0–R15 and state the special role of SP, LR, and PC.
6. Interpret the N, Z, C, and V flags of the APSR after an arithmetic operation.
7. Describe the layout of the Cortex-M 4 GB memory map and identify which region holds code, variables, and peripherals.
8. Explain little-endian byte ordering and distinguish the AHB and APB buses by the peripherals each one serves.

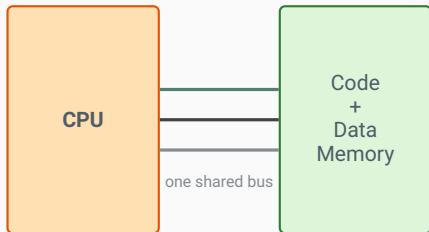
- 1 Why This Matters
- 2 **Architecture and Organization**
- 3 ARM and the Cortex Profiles
- 4 STM32: The Chip and the Boards
- 5 Cortex-M4 Programmer's Model
- 6 Design Challenge

Three Layers Called “Architecture”

Layer	What it specifies
Instruction SetArchitecture (ISA)	The language the processor understands: instruction formats, register set, addressing modes. Visible to an assembly programmer. <i>x86, ARM, RISC-V</i>
Microarchitecture	How that ISA is implemented in silicon: control signals, cache size, pipeline depth, ALU internals
Platformarchitecture	How the processor reaches the rest of the system: buses, memory controllers, DMA. <i>ARM's AMBA</i>

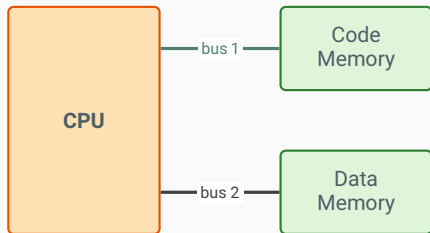
These layers are independent

One ISA can be built by many different microarchitectures. This is why code written for one ISA can run on physically different chips. It is the commercial foundation of the whole ARM ecosystem. The term “32-bit” can describe the ISA, the ALU width, or the address bus. On the Cortex-M4, these three meanings happen to be the same. But they are still three separate, independent facts.



- A **single** storage system holds both program and data.
- An instruction typically needs **two clock cycles**: one to fetch the instruction, another to fetch its data.
- Instruction and data cannot travel at the same time, since they share one bus.
- Simple and inexpensive, but the shared bus quickly becomes a bottleneck.

This shared-bus limit is called the **Von Neumann bottleneck**. The next slide shows an architecture built to remove it.



- **Two separate memories** for program and data, each with its own bus.
- With pipelining, an instruction can complete in **one cycle**: the next instruction is fetched while the current one reads its data.
- Most modern architectures are Harvard, or a close variant of it.

What the Cortex-M4 actually is

A **modified Harvard** machine. It has separate instruction and data buses (three AHB-Lite interfaces), but a **single unified address space**. You get the speed of Harvard, but you can still load a constant stored in flash with an ordinary load instruction.

CISC	RISC
Emerged when memory was expensive and compilers were primitive	Deliberately simplified so that hardware is fast and predictable
Complex hardware instruction decoder	Relies heavily on the compiler for optimisation
Variable-length instructions, multi-clock operations	Uniform-length instructions, single-cycle operations, simple pipelining
Memory-to-memory: arithmetic can operate directly on memory	Register-to-register: memory is reached only via explicit load and store
Dense code, but more cycles per instruction	More instructions, but far fewer cycles per instruction
e.g. Intel x86, AMD	e.g. ARM, RISC-V

Complexity does not disappear – it moves

RISC moves complexity out of the hardware decoder and into the compiler. The result is a set of simple instructions that pipeline cleanly, and behave in a predictable way.

Memory Access vs. Register Architecture

CISC – Memory-to-Memory

- Memory operations are embedded in arithmetic instructions.
- A single instruction can load from memory, compute, and write the result back.

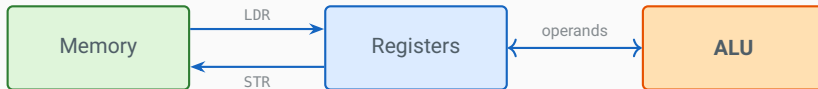
```
; x = x + 1  
ADD [eax], ebx
```

RISC – Register-to-Register

- Strict load-store architecture.
- Memory is accessed only by dedicated LDR and STR instructions.

```
; x = x + 1  
LDR r0, [r1]    ; load into register  
ADD r0, r0, r2  ; register arithmetic  
STR r0, [r1]    ; store back to memory
```

Key idea: the ALU never operates directly on memory. Data must first move into a register.

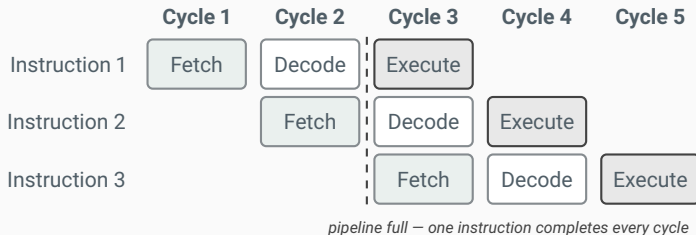


LDR → ALU operation → STR: this is the **only** way the ALU ever touches memory. Even $x = x + 1$; costs **three** instructions, never one. This pattern shapes every assembly example in Chapters 5 and 6.

Digital-electronics connection: registers sit only a few gate delays from the ALU. Memory sits across a bus, with address decoding and longer settling times. Forcing operands through registers first is what lets the clock run fast.

Doing fetch, decode, and execute strictly one after another **wastes hardware**. While the ALU is busy, the fetch unit sits idle, doing nothing.

A **pipeline** overlaps these stages instead, like doing laundry: you do not wait for one load to finish drying before starting to wash the next one.



- Each instruction still takes three cycles. But once the pipeline is full, **one instruction finishes every cycle**. The Cortex-M4 uses a three-stage pipeline with branch speculation.
- RISC instructions are simpler, so they are much easier to build a pipeline for.

The PC runs ahead While instruction 1 executes, instruction 2 is decoding, and instruction 3 has already been fetched. So the PC does not point at the instruction currently executing. This is why `MOV R0,` PC returns an address four bytes ahead of the current instruction.

Branches cost more When a branch is taken, the instructions already fetched behind it are wrong. The pipeline must be cleared and refilled. A three-stage pipeline loses only a few cycles. A long pipeline loses many more.

A deliberate design decision

A Cortex-A core uses a long pipeline for high peak speed. But a mispredicted branch is expensive there, and how expensive depends on the data. The Cortex-M gives up some peak speed on purpose, so its worst-case timing stays close to its average-case timing. Chapter 1 called this property essential for embedded systems.

$$\text{CPU time} = \text{Instruction Count} \times \text{Cycles per Instruction} \times \text{Clock period}$$

Term	RISC effect	Why
Instruction count	Higher	More, simpler instructions per task
Cycles per instruction	Lower	Each instruction finishes in one cycle
Clock period	Lower	A simple decoder permits a faster clock

The product of all three terms is what matters in the end. RISC improves two of the three terms, which is why it usually wins.

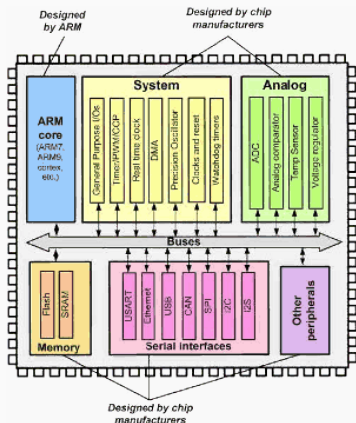
- 1 Why This Matters
- 2 Architecture and Organization
- 3 ARM and the Cortex Profiles**
- 4 STM32: The Chip and the Boards
- 5 Cortex-M4 Programmer's Model
- 6 Design Challenge

- **1980s** — Acorn Computers in Cambridge builds the **BBC Micro** around the 6502, for the BBC's computer-literacy project.
- **1985** — Sophie Wilson and Steve Furber design a new CPU: the **Acorn RISC Machine**. The ARM1 has only 24 800 transistors and uses under 100 mW.
- **1990** — ARM Ltd is spun out, jointly owned by Acorn, Apple, and VLSI. "Acorn" becomes **Advanced RISC Machines**.
- **1993** — The Apple Newton ships with an ARM610. The product itself sells poorly, but the partnership pushes ARM toward a pure licensing model.
- **2000s** — Mobile phones make ARM the default choice: low power, good enough performance, and easy to license.
- **Today** — The most widely used ISA on Earth, used in everything from tiny sensors to Apple's M-series chips and Amazon's Graviton servers.

The pattern worth noticing

ARM was designed to be small because Acorn was a small company with a limited budget. That early constraint later became the reason ARM came to dominate the embedded and mobile markets. This is the same cost-and-power argument as in Chapter 1.

ARM Does Not Make Chips Why there is an ST logo on an ARM core



ARM is **fabless**: it designs processor cores and licenses the design to others. It owns no chip factory itself.

- **Core licence** — A chip maker puts an ARM core into its own design, then adds its own memory, peripherals, and analog circuits (ST, NXP, Microchip, Nordic, ...).
- **Architecture licence** — A company designs its own core that still runs the ARM instruction set. Fewer than ten such licences exist. Apple's M-series is one example.

On your bench: ARM designed the **Cortex-M4 core**. **ST** added the flash, SRAM, GPIO, timers, ADC, and UART around it. The finished product is the **STM32F446RE**.

The hierarchy to keep straight

ARM defines the instruction-set architecture and licenses cores. **Cortex** names families of processor cores built around it. For embedded systems, the family we use is **Cortex-M**.

	Cortex-A	Cortex-R	Cortex-M
Profile	Application — highest performance	Real-time — fast response	Microcontroller — smallest, lowest power
OS	Linux, Android	RTOS	Bare metal / RTOS
Instruction set	64-bit, 32-bit, Thumb	32-bit, Thumb	Thumb only
Interrupts	Software-managed	Deterministic, software-managed	Hardware-managed
Pipeline	Long	Long to medium	Short
Cache	Multi-level	Yes	None
Found in	Phones, tablets, servers	Storage, industrial, automotive	IoT, sensors, control

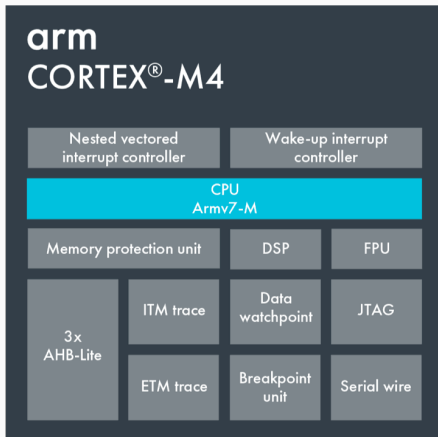
Cortex-M trades peak speed for predictable, low-power execution

A short pipeline and no cache mean an instruction takes exactly the time the documentation says it takes. This is the property we said was essential for embedded systems.

So where does the Cortex-M4 fit inside the Cortex-M family? That is next.

Processor	Description
Cortex-M0	Very small (from 12K gates), lowest cost, ultra-low power
Cortex-M0+	Most energy-efficient; adds single-cycle I/O and vector-table relocation
Cortex-M3	Rich instruction set, hardware divide, MAC, full debug and trace
Cortex-M4	Everything in the M3, plus DSP (SIMD, fast MAC) and an optional single-precision FPU
Cortex-M7	High performance, dual-issue pipeline, optional double-precision FPU, cache and TCM
Cortex-M23	ARMv8-M Baseline with TrustZone; ultra-low-power secure IoT
Cortex-M33	ARMv8-M Mainline with TrustZone, optional DSP/FPU; widely used secure MCU core
Cortex-M35P	Like M33 but with built-in physical tamper resistance
Cortex-M52	Smallest core with Arm Helium (MVE); efficient ML/DSP for cost-sensitive designs
Cortex-M55	First Helium core; strong scalar + vector (ML/DSP) performance
Cortex-M85	Highest-performance Cortex-M; superscalar + Helium, strongest ML/DSP and security features

The **F** in STM32F446RE's core name — Cortex-M4**F** — shows that a floating-point unit is fitted. Your board can therefore do single-precision floating-point arithmetic in hardware.

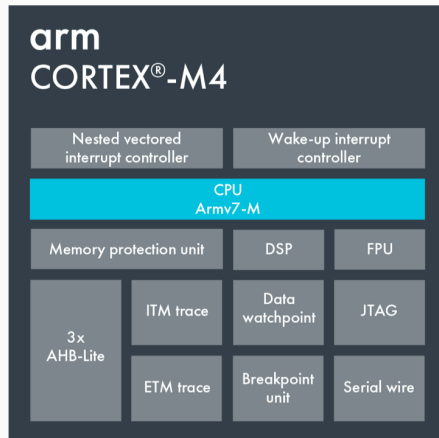


What each block is for

- **CPU (Armv7-M)** — the datapath from Chapter 1, now with 16 registers.
- **NVIC** — Nested Vectored Interrupt Controller. Interrupts are managed in hardware (Chapter 9).
- **DSP / FPU** — SIMD operations, fast multiply-accumulate, and single-precision floating-point.
- **MPU** — Memory Protection Unit.
- **3 × AHB-Lite** — the separate instruction and data buses that give the core its modified-Harvard design.
- **JTAG / Serial Wire, ITM/ETM** — the debug and trace hardware your ST-LINK talks to.

Cortex-M4 Features The specification, in one place

Architecture	ARMv7E-M
ISA	Thumb-2
Pipeline	3-stage + branch speculation
Bus interface	3 × AMBA AHB-Lite
DSP	Single-cycle 16/32-bit MAC Single-cycle dual 16-bit MAC 8/16-bit SIMD arithmetic Hardware divide (2–12 cycles)
Floating point	Single precision, IEEE 754
Interrupts	Nested Vectored Interrupt Controller Non-maskable interrupt (NMI) 1 to 240 physical interrupts 8 to 256 priority levels
HLL support	Designed to be programmed fully in C



Every row above has already appeared in this chapter, except the last one. That last row is the reason Chapter 4 is written in C, not assembly.

Memory-Mapped I/O The idea the rest of the course rests on

There is no special instruction for I/O

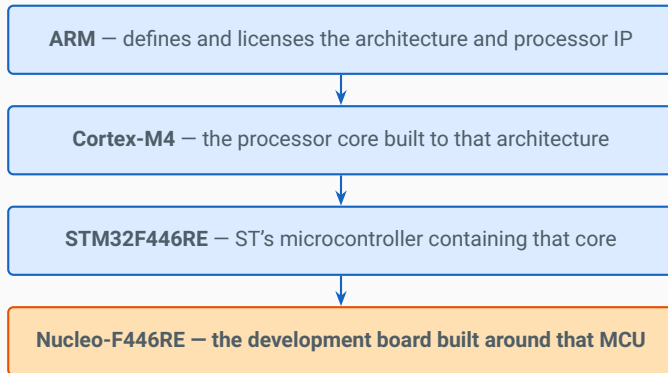
A peripheral register is just an address. You turn an LED on with the same STR instruction you use to write to a variable. Only the address is different.

What you write in C	What it becomes	Where it lands
<code>c = a + b;</code>	<code>STR R2, [R3]</code>	SRAM, 0x2000 xxxx
<code>GPIOA->ODR = (1<<5);</code>	<code>STR R2, [R3]</code>	peripheral , 0x4002 0014

Same instruction, same registers, different address. And one of those addresses lights a real LED on PA5. That is the whole trick. Chapter 8 is where you put it to use.

- 1 Why This Matters
- 2 Architecture and Organization
- 3 ARM and the Cortex Profiles
- 4 STM32: The Chip and the Boards**
- 5 Cortex-M4 Programmer's Model
- 6 Design Challenge

Four names are easy to confuse: ARM, Cortex-M4, STM32, Nucleo. Each name is a different layer of the same stack.



Every slide from here on sits at one of these four levels. Keeping them separate is the best way to avoid confusing “ARM,” “the chip,” and “the board.”

What Is an STM32?

STM32

STMicroelectronics' family of **32**-bit microcontrollers, each built around a licensed **ARM Cortex-M** core.

ARM supplies the core

- The Cortex-M4 datapath, registers, and instruction set: everything covered so far in this chapter.
- The same core, across every vendor's chips.

So Cortex-M4 code can move between different chips, at the core level.

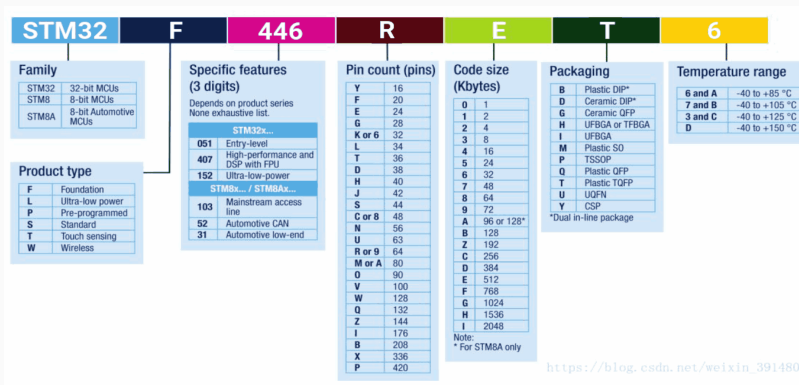
The family covers many market needs: **F** (general purpose), **L** (low power), **G** (advanced general purpose), **H** (high performance), **WB** (wireless), **U** (ultra-low power). We use an **STM32F4**.

ST adds everything else

- Flash and SRAM, in specific sizes
- Peripherals: GPIO, timers, ADC, DAC, UART, SPI, I²C, CAN, USB
- Clock tree, power management, packaging

These parts are **not** portable. They are ST's own design. Chapters 8–13 focus on programming them.

Reading the Part Number Every character means something



STM32
32-bit MCU

F
Foundation

446
high perf. +
DSP with FPU

R
64 pins

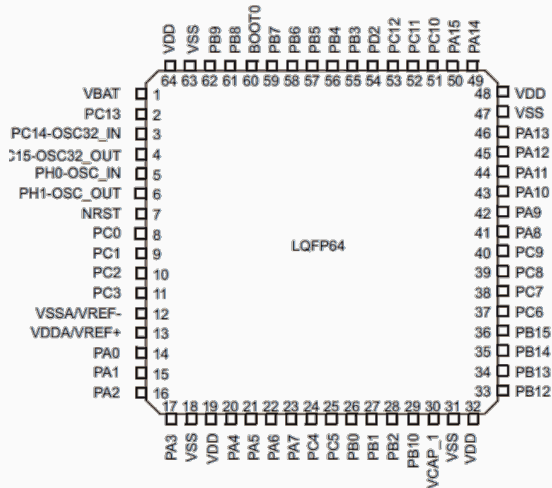
E
512 KB
flash

T
plastic
TQFP

6
-40 to
+85 °C

A part number is a specification. It is not a serial number.

The Chip Itself STM32F446RE in LQFP64

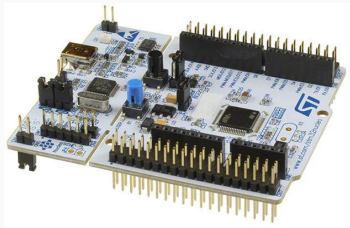


STM32F446RE

- Core: ARM Cortex-M4
- Clock: up to 180 MHz
- Flash: 512 KB
- SRAM: 128 KB
- FPU and DSP: single-precision floating-point and DSP instructions for real-time work
- Rich peripheral set: GPIO, timers, UART, I²C, SPI, CAN, USB

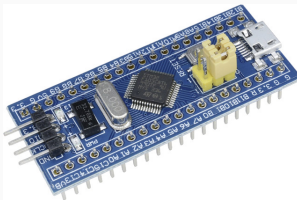
R = 64 pins and T = LQFP, exactly as the part number said two slides ago.

STM32 Boards The three you will actually use



Nucleo-F446RE

- Our running board all semester
- ST-LINK built in – no separate debugger needed
- Arduino Uno pinout + morpho headers



Blue Pill

- Not made by ST; one of many third-party boards
- Cheapest way to get a Cortex-M4 on a breadboard
- Debugger **not** included – needs the ST-LINK

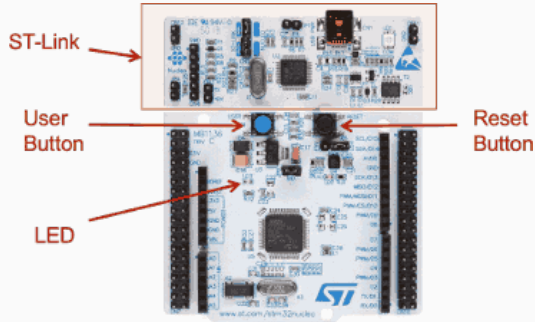


ST-LINK V2

- External programmer/debugger over SWD
- What the Blue Pill needs; the Nucleo has one built in
- Same tool, same workflow, either board

Same Cortex-M4 core, same ISA, but very different boards.

The Nucleo-F446RE Board What ST adds around the chip

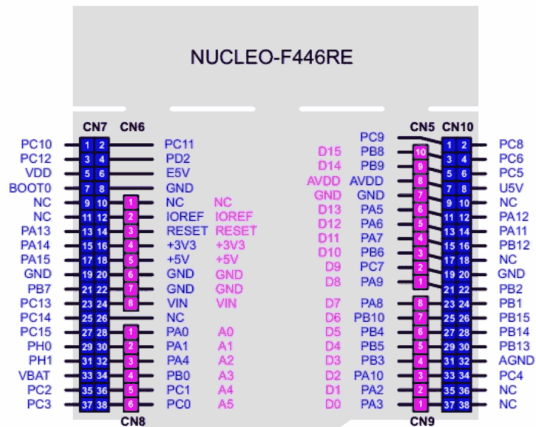


The Nucleo board adds four useful features that make experiments easy:

- **ST-Link debugger** – the detachable top section. One USB cable supplies power, programming, and debugging. No separate programmer is needed.
- **User button and LED** – PC13 and PA5. Enough to check a program works, with no external wiring.
- **Arduino connector** – accepts standard off-the-shelf shields.
- **Morpho connector** – brings out every STM32 pin, including the ones the Arduino header cannot reach.

Full details are in the *STM32 Nucleo-64 Boards User Manual (UM1724)*.

Nucleo-F446RE Pinout



Two connector systems share the board:

- **Arduino** headers (CN5–CN10, magenta) – D0–D15, A0–A5. Accept standard shields.
- **Morpho** headers (CN7, CN10, blue) – every STM32 pin, including the ones the Arduino header leaves out.

Pins used throughout the course:

PA5 – user LED (Arduino D13)

PC13 – user button

- 1 Why This Matters
- 2 Architecture and Organization
- 3 ARM and the Cortex Profiles
- 4 STM32: The Chip and the Boards
- 5 Cortex-M4 Programmer's Model**
- 6 Design Challenge

We move from processor *architecture* to the processor's *programmer's model*.

What can your C or assembly program actually see?

This is the question we opened the chapter with: “*This detailed view is called the programmer’s model – the part of the hardware that software can actually see.*” We can now describe that model in detail: registers, flags, memory, and peripherals.

The Complete Register Set

Core registers in focus; special registers for reference

General-purpose

R0
R1
R2
R3
R4
R5
R6
R7
R8
R9
R10
R11
R12
R13 (SP)
R14 (LR)
R15 (PC)

Special registers (Ch. 9)

PSR
PRIMASK
FAULTMASK
BASEPRI
CONTROL

Stack pointer banking

PSP
MSP

Register	Name	Role
R0–R12	General purpose	Any data or address. R0–R3 carry the first four function arguments; R0 holds the return value
R13	SP Stack Pointer	Points to the top of the stack in SRAM. Banked as MSP / PSP
R14	LR Link Register	Holds the return address; written automatically by BL
R15	PC Program Counter	Address of the next instruction. Writing it performs a branch
xPSR	Status	Condition flags and execution state

Sixteen is a deliberately small register file

MIPS and RISC-V give the programmer 32 registers. ARM gives up a little compiler flexibility for a smaller, faster, lower-power register file. This fits both the load-store idea and the needs of embedded systems.

SP is really two banked registers, MSP and PSP, chosen by CONTROL. The mask registers (PRIMASK, FAULTMASK, BASEPRI) control interrupts. Not every Cortex-M core has them; Chapter 9 comes back to this.

The surprise

MOV R0, PC does **not** give you the address of that MOV. It gives that address **+ 4**.

Why — this is just the pipeline

- By the time an instruction executes, the next ones have **already been fetched**.
- “The current instruction” has no clear meaning on a pipelined machine.
- So the PC runs ahead. On the Cortex-M4, it runs exactly one instruction ahead.

Why you care

- You will meet this in a debugger.
- It looks exactly like an off-by-one bug in your own code.
- It is not. It is the architecture.

Three lines you could have written in your first programming course:

```
int a = 10;  
int b = 20;  
int c = a + b;
```

The Cortex-M4 is a load-store machine, so these values must travel a fixed path before any arithmetic can happen:



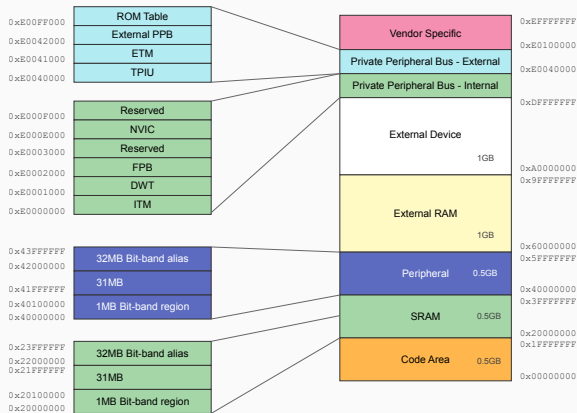
The register set is not an abstract list. It is the set of places where your variables must **physically be** before the ALU can touch them.



Bit	Flag	Set when
31	N Negative	Bit 31 of the result is 1 (negative when interpreted as signed)
30	Z Zero	The result is exactly zero (set by a CMP of equal values)
29	C Carry	Unsigned addition produced a carry, or unsigned subtraction did <i>not</i> borrow
28	V Overflow	A signed operation exceeded the representable range

This is how a processor makes decisions: the C statement `if (x == y)` works by subtracting, throwing away the difference, and testing the **Z** flag. **C** is the carry-out of the final full adder. **V** is the XOR of the carries into and out of the MSB. These are circuits you have already built.

The Generic ARMv7-M Memory Map



This is the map defined by ARM.

ARM defines a **standardized** address space shared by every Cortex-M core. This is what lets code move between chips from different vendors.

The space is 4 GB wide, split into sub-regions, each with its own separate role.

Base	Region
0x0000 0000	Code – 512 MB
0x2000 0000	SRAM – 512 MB
0x4000 0000	Peripheral – 512 MB
0x6000 0000	External RAM – 1 GB
0xA000 0000	External device – 1 GB
0xE000 0000	Private peripheral bus

Code area (512 MB)

- Maps to Flash: program code and constant data
- Also holds the bootloader and the **interrupt vector table**
- Code executes directly from Flash

SRAM (512 MB)

- Variables, the stack, dynamic data
- Faster to access than Flash

Peripheral (512 MB)

- The memory-mapped registers of every on-chip peripheral – GPIO, timers, ADC, DAC, UART, SPI, I²C
- Each peripheral gets its own address range

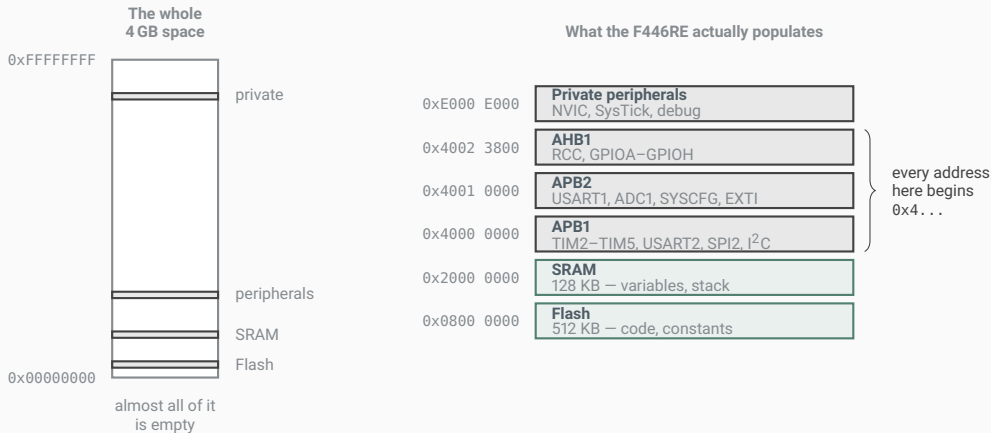
Private peripheral bus

- System peripherals: **NVIC**, SysTick, MPU
- Debug and trace: DWT, ITM, FPB

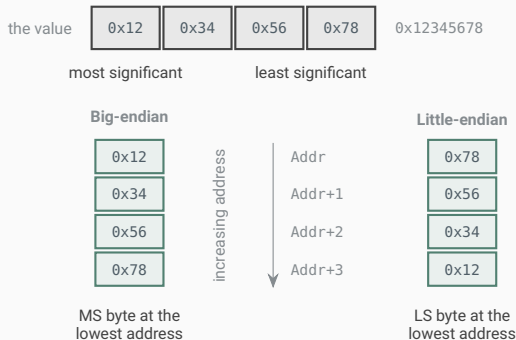
On your STM32F446RE

512 KB of Flash and 128 KB of SRAM fill only a small part of the regions above. The rest of the map is **reserved**, not missing. Reading an address with nothing there causes a HardFault.

The 4 GB Memory Map The STM32F446RE inside it



Now let's see where the real STM32 resources live. ARM **fixes** this layout for every Cortex-M device. This is why the NVIC sits at the same address on every vendor's chip. Reading an empty region causes a **HardFault**. This is why a wild pointer crashes loudly, instead of failing quietly.



Both conventions store the **same four bytes**. They only differ in which end sits at the lowest address. The Cortex-M4 is **little-endian**, like x86, so raw memory dumps look the same on the target and on a typical PC.

Next: Chapter 3 — Memory, Addressing and Pointers

If a peripheral is just an address, then a pointer is how C reaches it.

- 1 Why This Matters
- 2 Architecture and Organization
- 3 ARM and the Cortex Profiles
- 4 STM32: The Chip and the Boards
- 5 Cortex-M4 Programmer's Model
- 6 Design Challenge

We now use this chapter's performance model to make a processor-selection decision.

The brief

You are choosing a processor for a motor controller. It must run a control loop 20 000 times per second. Each pass through the loop needs about 180 instructions.

Using the CPU-time equation, find the minimum clock frequency needed if the processor averages 1.3 cycles per instruction. Then calculate what fraction of the CPU that loop uses at 16 MHz, at 48 MHz, and at 180 MHz. Decide which of the three frequencies you would choose, and explain the safety margin you leave. Finally, state what you would need to measure to check whether the 1.3 CPI assumption is realistic.

Questions?

Introduction to ARM Cortex-M Microprocessors — Chapter 2